ML Theory :

Library	Purpose
Pandas	Reads and handles dataset
train_test_split	Splits data into training and testing
StandardScaler	Normalizes numerical data
MLPClassifier	Builds a Neural Network
accuracy_score	Calculates accuracy
confusion_matrix	Evaluates predictions

Viva: What is Scikit-learn?

Scikit-learn is a Python library used for machine learning. It provides algorithms for classification, regression, clustering and data preprocessing.

Here:

- X = Independent variables (input features).
- y = Dependent variable (output/target).

Why remove these columns?

Column	Reason

Exited	This is our target, not an input
RowNumber	Just a row index
CustomerId	Unique customer identification
Surname	Not required for this basic prediction model

Remember: `Exited` is the column we want to predict.

- 0 = Customer stays.
- 1 = Customer leaves.

Important parameters:

- `test_size=0.2`: 20% data is reserved for testing.
- `random_state=42`: Keeps the split reproducible.
- `stratify=y`: Maintains the proportion of customers who leave and stay in both datasets.

Accuracy: Measures the percentage of correctly predicted records.

$$Accuracy = \frac{CorrectPredictions}{TotalPredictions} \times 100$$

Confusion Matrix: Shows correct and incorrect predictions.

Question

Answer

What is MLP?

Multi-Layer Perceptron

What is MLPClassifier?

A Neural Network classification algorithm in Scikit-learn

What is a neuron?

A basic processing unit of a Neural Network

What is a hidden layer?

A layer between input and output that learns patterns

What is an epoch?

One complete pass through the training dataset

What is overfitting?

When a model performs well on training data but poorly on unseen data

What is early stopping?

A technique that stops training when validation performance stops improving

Why use StandardScaler?

To standardize numerical features

Why use get_dummies?

To convert categorical data into numerical form

What is binary classification?

Predicting one of two possible classes

What is random_state?

Ensures reproducible random operations

What is max_iter?

Maximum number of training iterations

Prac 2

ML-2: Email Spam Classification Using KNN and SVM

Aim: To classify emails into two categories using K-Nearest Neighbors (KNN) and Support Vector Machine (SVM) algorithms and compare their performance.

- Normal State: Not Spam (0)
- Abnormal State: Spam (1)

Let's understand the theory, each line of your code, and important viva questions.

1. Theory

What is Email Spam Classification?

Email spam classification is the process of identifying whether an email is genuine or unwanted.

For example:

Email	Classification
Your college exam timetable is released	Not Spam
You have won ₹10 lakh! Click here	Spam
Meeting scheduled for tomorrow	Not Spam
Claim your free prize now	Spam

This is a Binary Classification problem because there are only two possible outputs:

- 0 = Normal / Not Spam
- 1 = Abnormal / Spam

What is K-Nearest Neighbors (KNN)?

KNN stands for K-Nearest Neighbors. It is a supervised machine learning algorithm used for classification and regression.

KNN classifies a new data point by looking at its K nearest neighboring data points.

How does KNN work?

1. Select the value of K.
2. Calculate the distance between the new data point and existing data points.
3. Find the K nearest neighbors.
4. Check the majority class among those neighbors.
5. Assign the majority class to the new data point.

Class AClass BPredictionClass A

k = 5: Class A wins 3–2; the point is Class A.

k

Give feedback

Example:

Suppose K = 5 and we want to classify a new email.

Among its 5 nearest neighbors:

- 3 emails are Spam.
- 2 emails are Not Spam.

Therefore, KNN classifies the new email as Spam, because Spam has the majority vote.

What is the distance formula?

KNN commonly uses Euclidean distance:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

It measures the distance between two data points.

What is Support Vector Machine (SVM)?

SVM stands for Support Vector Machine. It is a supervised machine learning algorithm used mainly for classification.

SVM finds the best decision boundary (hyperplane) that separates different classes.

For example, it separates Spam emails from Not Spam emails.

What is SVM and How Does It Work? | by Tech & Tales | AI monks.io | Medium

medium.com

Important terms in SVM:

1. Hyperplane: A decision boundary that separates two classes.
2. Support Vectors: Data points closest to the hyperplane.
3. Margin: Distance between the hyperplane and the nearest data points of both classes.
4. Kernel: A function that helps SVM handle data that cannot be separated using a straight boundary.

Your code uses a linear kernel, which creates a linear decision boundary.

2. Difference between KNN and SVM

KNN	SVM
K-Nearest Neighbors	Support Vector Machine
Distance-based algorithm	Hyperplane-based algorithm
Uses majority voting	Uses an optimal decision boundary
Lazy learning algorithm	Eager learning algorithm
Sensitive to feature scaling	Sensitive to feature scaling

Prediction can be slower for large datasets	Prediction can be efficient after training
Simple to understand	More mathematical

3. Code explanation line by line

Step 1: Import libraries

```
None
import pandas as pdfrom sklearn.model_selection import train_test_splitfrom sklearn.preprocessing import StandardScalerfrom sklearn.neighbors import KNeighborsClassifierfrom sklearn.svm import SVCfrom sklearn.metrics import accuracy_score
```

Library	Purpose
Pandas	Reads and handles dataset
train_test_split	Splits training and testing data
StandardScaler	Normalizes data
KNeighborsClassifier	Implements KNN

SVC	Implements SVM
accuracy_score	Calculates accuracy

Viva: What is SVC?

SVC stands for Support Vector Classifier. It is Scikit-learn's implementation of the Support Vector Machine classification algorithm.

Step 2: Read the dataset

```
None

df = pd.read_csv('emails.csv')
```

- Reads the `emails.csv` file.
- Stores the data in a DataFrame called `df`.

Step 3: Separate features and target

```
None

y = df.iloc[:, -1]X = df.iloc[:, :-1].drop(    columns=['Email No.'],    errors='ignore')
```

This is an important part of your code.

Understanding `iloc`:

`iloc` is used to select rows and columns using their numerical positions.

- `df.iloc[:, -1]` selects the last column.
- `df.iloc[:, :-1]` selects all columns except the last column.

Here:

- `X` = Input features (email data).
- `y` = Output/target (Spam or Not Spam).

Why remove Email No.?

Email No. is just an identification number. It does not help the model identify spam.

Step 4: Split the dataset

```
None

X_train, X_test, y_train, y_test = train_test_split(    X, y,
test_size=0.2,    random_state=42,    stratify=y)
```

The dataset is divided into:

- 80% training data.
- 20% testing data.

Suppose the dataset has 5,000 emails:

Dataset	Number of emails
Training	4,000
Testing	1,000
Total	5,000

Why do we split data?

Training data teaches the model, while testing data checks how accurately the model predicts unseen emails.

Step 5: Normalize the data

```
None

scaler = StandardScaler()X_train =
scaler.fit_transform(X_train)X_test = scaler.transform(X_test)
```

Why is normalization required?

Suppose one feature has values between 0 and 10, while another has values between 0 and 1,000.

KNN calculates distances, so features with larger scales may dominate the distance calculation.

StandardScaler brings the features to a comparable scale.

Remember:

- `fit_transform()` learns and transforms training data.
- `transform()` transforms testing data using the same training statistics.

Step 6: Initialize and train KNN

```
None

knn = KNeighborsClassifier(n_neighbors=5)knn.fit(X_train,
y_train)knn_pred = knn.predict(X_test)
```

Explanation:

- `KNeighborsClassifier()` creates the KNN model.
- `n_neighbors=5` means the algorithm considers 5 nearest neighbors.
- `fit()` trains the model.
- `predict()` predicts whether each email is spam or not spam.

Viva: What happens if we increase K?

Increasing K considers more neighbors. It can make the decision boundary smoother, but an excessively large K may cause underfitting.

Step 7: Initialize and train SVM

```
None

svm = SVC(kernel='linear')svm.fit(X_train, y_train)svm_pred =
svm.predict(X_test)
```

Explanation:

- `SVC()` creates the SVM classifier.
- `kernel='linear'` uses a linear decision boundary.

- `fit()` trains the model.
- `predict()` predicts the email category.

Viva: What is a kernel?

A kernel is a function used by SVM to determine how data points are separated. Common kernels are:

- Linear
- Polynomial
- RBF
- Sigmoid

Step 8: Calculate accuracy and compare performance

```
None

print('KNN Accuracy:',      accuracy_score(y_test,
knn_pred))print('SVM Accuracy:',      accuracy_score(y_test,
svm_pred))
```

This calculates the accuracy of both algorithms.

Accuracy formula:

$$Accuracy = \frac{CorrectPredictions}{TotalPredictions} \times 100$$

Example (illustrative):

Algorithm	Accuracy
KNN	92%
SVM	96%

In this example, SVM performs better because it has higher accuracy. Your actual results may differ.

4. How to analyze performance?

The question asks you to analyze the performance of KNN and SVM.

After executing the code, compare their accuracy.

For example:

- If KNN accuracy is 90%, it correctly classifies 90% of test emails.
- If SVM accuracy is 95%, it correctly classifies 95% of test emails.
- Therefore, SVM performs better in terms of accuracy in this example.

One improvement to your code: Accuracy alone is not enough for spam classification. We should also check precision, recall and F1-score.

```
None
from sklearn.metrics import classification_reportprint("KNN
Performance:")print(classification_report(y_test,
knn_pred))print("SVM
Performance:")print(classification_report(y_test, svm_pred))
```

Metric	Meaning
Accuracy	Overall correct predictions
Precision	How many predicted spam emails were actually spam
Recall	How many actual spam emails were correctly identified
F1-score	Balance between precision and recall

For spam detection, recall is important because low spam recall means many spam emails are incorrectly classified as normal.

5. Important viva questions and answers

Q1. What is spam?

Spam is an unwanted or unsolicited email, usually sent for advertising, fraud or spreading malicious links.

Q2. What is binary classification?

It is a classification technique in which the output has only two classes, such as Spam and Not Spam.

Q3. What is supervised learning?

Supervised learning is a type of machine learning in which the model learns from labelled training data.

Q4. What is KNN?

KNN is a supervised learning algorithm that classifies a new data point based on the majority class of its nearest neighbors.

Q5. What does K represent in KNN?

K represents the number of nearest neighbors considered for classification.

Q6. Why do we use K=5?

It considers five nearest neighbors and uses majority voting. It is a commonly used starting value, but the best K should be selected through experimentation.

Q7. What is Euclidean distance?

It is the straight-line distance between two data points.

Q8. What is SVM?

SVM is a supervised learning algorithm that finds the best hyperplane to separate different classes.

Q9. What are support vectors?

Support vectors are the data points closest to the decision boundary. They help determine the position of the hyperplane.

Q10. What is a hyperplane?

A hyperplane is a decision boundary used to separate different classes.

Q11. What is a kernel in SVM?

A kernel helps SVM classify data by determining the decision boundary. Examples include Linear, Polynomial and RBF.

Q12. Why do we use StandardScaler?

To standardize features so that differences in their numerical scales do not negatively affect the algorithms.

Q13. What is the difference between KNN and SVM?

KNN uses distance and majority voting, whereas SVM uses a hyperplane to separate classes.

Q14. Which algorithm is better, KNN or SVM?

The algorithm with better performance on unseen test data is preferred. We compare accuracy, precision, recall and F1-score.

Q15. What is overfitting?

Overfitting occurs when a model learns training data too closely and performs poorly on unseen data.

6. Quick revision for practical exam

ML-2 execution flow

1. Read dataset – `pd.read_csv()`
2. Separate features and target – `iloc`
3. Remove Email No. – `drop()`
4. Split data – `train_test_split()`
5. Normalize – `StandardScaler()`
6. Build KNN – `KNeighborsClassifier(n_neighbors=5)`
7. Build SVM – `SVC(kernel='linear')`
8. Train – `fit()`
9. Predict – `predict()`
10. Compare – `accuracy_score()`

7. Test your understanding

Q1. What does n_neighbors=5 mean in KNN?

5 hidden layers

5 nearest neighboring data points

5 training datasets

Q2. Why is StandardScaler especially important for KNN?

Because KNN uses distance calculations

Because it removes spam emails

Because it increases the number of features

Q3. What does kernel='linear' do in SVM?

Uses five neighbors

Creates a linear decision boundary

Normalizes the dataset

Check answers

One important thing to remember: KNN = Distance + Neighbors + Voting. SVM = Hyperplane + Margin + Support Vectors.

This is the core difference your external examiner is most likely to ask about.

Prac 3:

ML-3: Diabetes Prediction Using K-Nearest Neighbors (KNN)

Aim: To implement the K-Nearest Neighbors algorithm on the Diabetes dataset and calculate the confusion matrix, accuracy, error rate, precision and recall.

1. Theory

What is Diabetes Prediction?

Diabetes prediction is a machine learning application used to predict whether a person has diabetes based on medical information.

For example, a person's:

- Glucose level
- Blood pressure
- Insulin level
- BMI
- Age
- Pregnancies

are given as input to the model.

The model predicts whether the person is diabetic or non-diabetic.

What is K-Nearest Neighbors (KNN)?

KNN stands for K-Nearest Neighbors. It is a supervised machine learning algorithm used for classification and regression.

It predicts the class of a new data point by checking the classes of its nearest neighboring data points.

Working of KNN:

1. Select the value of K.
2. Calculate the distance between data points.
3. Find the K nearest neighbors.
4. Count the classes of the nearest neighbors.
5. Assign the majority class as the final prediction.

Class AClass BPredictionClass A

k = 5: Class A wins 3–2; the point is Class A.

k

Give feedback

Example

Suppose K = 5.

For a new patient's medical data, the 5 nearest neighbors are:

- 3 patients are diabetic.
- 2 patients are non-diabetic.

Therefore, KNN predicts that the new patient is diabetic, because the majority of neighbors belong to the diabetic class.

What is the target variable?

The target column is `Outcome`.

Outcome	Meaning
0	Non-diabetic
1	Diabetic

This is a Binary Classification problem because there are only two possible outputs.

2. Understanding the dataset

The Diabetes dataset contains medical information about patients.

Important columns include:

Feature	Meaning
Pregnancies	Number of times pregnant
Glucose	Blood glucose concentration
BloodPressure	Blood pressure
SkinThickness	Skin thickness measurement
Insulin	Insulin level
BMI	Body Mass Index
DiabetesPedigreeFunction	Family history-related diabetes risk indicator
Age	Patient's age
Outcome	0 = Non-diabetic, 1 = Diabetic

`Outcome` is the target variable that we want to predict.

3. Code explanation line by line

Step 1: Import libraries

```
None

import pandas as pdfrom sklearn.model_selection import
train_test_splitfrom sklearn.preprocessing import
StandardScalerfrom sklearn.neighbors import
KNeighborsClassifierfrom sklearn.metrics import (
confusion_matrix,    accuracy_score,    precision_score,
recall_score)
```

Library	Purpose
Pandas	Reads and handles the dataset
train_test_split	Divides the dataset
StandardScaler	Normalizes data
KNeighborsClassifier	Implements KNN
confusion_matrix	Calculates correct and incorrect predictions
accuracy_score	Calculates accuracy
precision_score	Calculates precision
recall_score	Calculates recall

Step 2: Read the dataset

```
None

df = pd.read_csv('diabetes.csv')print(df.head())
```

- `read_csv()` reads the CSV file.
- `df` stores the dataset.
- `head()` displays the first five records.

Step 3: Separate features and target

```
None

X = df.drop('Outcome', axis=1)y = df['Outcome']
```

Here:

- `X` contains the input features.
- `y` contains the target/output.

Why do we drop Outcome from X?

Because Outcome is the value we want the model to predict. If we include it in X, the model would get access to the answer during training.

What is axis=1?

- `axis=0` means rows.
- `axis=1` means columns.

Therefore, `axis=1` removes the Outcome column.

Step 4: Split the dataset

```
None

X_train, X_test, y_train, y_test = train_test_split(    X, y,
test_size=0.2,    random_state=42,    stratify=y)
```

The dataset is divided into two parts.

Dataset	Percentage
Training data	80%
Testing data	20%

If the dataset contains 768 records:

- Training data = Approximately 614 records.
- Testing data = Approximately 154 records.

Important parameters:

- `test_size=0.2`: Uses 20% data for testing.
- `random_state=42`: Makes splitting reproducible.
- `stratify=y`: Maintains the proportion of diabetic and non-diabetic patients.

Step 5: Normalize the data

```
None

scaler = StandardScaler()X_train =
scaler.fit_transform(X_train)X_test = scaler.transform(X_test)
```

Why do we normalize?

Suppose:

- Glucose = 150
- Age = 45
- BMI = 30

These features have different numerical ranges.

KNN calculates distances between data points. Therefore, features with larger numerical values can dominate the distance calculation.

StandardScaler brings the features to a comparable scale.

Formula:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- X = Original value
- μ = Mean
- σ = Standard deviation

Remember:

Function	Meaning
fit_transform()	Learns statistics and transforms training data
transform()	Applies the same scaling to testing data

Step 6: Create and train the KNN model

```
None

model = KNeighborsClassifier(n_neighbors=5)model.fit(X_train, y_train)
```

Explanation:

- `KNeighborsClassifier()` creates the KNN model.
- `n_neighbors=5` means it considers five nearest neighbors.
- `fit()` trains the model using training data.

Why K = 5?

The algorithm checks the five nearest patients and predicts the majority class.

K = 5 is a commonly used starting value. We can experiment with different K values to find the best performance.

Step 7: Predict the output

```
None

pred = model.predict(X_test)
```

The trained model predicts the outcome for testing patients.

- 0 = Non-diabetic.
- 1 = Diabetic.

Now we compare the predicted values with the actual values.

4. Evaluation metrics – Very important

Your practical specifically asks for five evaluation measures. Understand these properly because they are frequently asked in viva.

A. Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model.

It compares actual values with predicted values.

A Secret Weapon: Power of Confusion Matrix in Machine Learning | by Devansh Sharma | Medium

dstechbytes.medium.com

Term	Meaning
True Positive (TP)	Patient has diabetes and model predicts diabetes
True Negative (TN)	Patient does not have diabetes and model predicts non-diabetes

False Positive (FP)	Patient does not have diabetes but model predicts diabetes
False Negative (FN)	Patient has diabetes but model predicts non-diabetes

B. Accuracy

Accuracy tells us how many predictions are correct out of all predictions.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Example: If the model correctly predicts 140 out of 154 patients:

\[Accuracy=\frac{140}{154}\times100=90.91\% \]

C. Error Rate

Error rate tells us the percentage of incorrect predictions.

$$Error\ Rate = 1 - Accuracy$$

For example, if accuracy is 90%:

\[Error\ Rate=1-0.90=0.10=10\% \]

D. Precision

Precision tells us how many patients predicted as diabetic are actually diabetic.

$$Precision = \frac{TP}{TP+FP}$$

Example:

If the model predicts 50 patients as diabetic, and 40 actually have diabetes:

\[Precision=\frac{40}{50}=0.80=80\% \]

E. Recall

Recall tells us how many actual diabetic patients are correctly identified by the model.

$$Recall = \frac{TP}{TP+FN}$$

Example:

If 60 patients actually have diabetes and the model correctly identifies 45:

\[Recall=\frac{45}{60}=0.75=75\% \]

Important difference between Precision and Recall:

Precision	Recall
Focuses on correct positive predictions	Focuses on finding all actual positive cases
Reduces false positives	Reduces false negatives
Of predicted diabetic patients, how many are actually diabetic?	Of actual diabetic patients, how many did we identify?

For diabetes prediction, recall is especially important because we want to identify as many actual diabetic patients as possible.

5. Step 8: Calculate all evaluation metrics

This is the final part of your practical.

```
None
accuracy = accuracy_score(y_test, pred)print('Confusion Matrix:\n', confusion_matrix(y_test, pred))print('Accuracy:',
```

```
accuracy)print('Error Rate:', 1 - accuracy)print('Precision:',
precision_score(y_test, pred, zero_division=0))print('Recall:',
recall_score(y_test, pred, zero_division=0))
```

Explanation:

- `confusion_matrix()` displays TP, TN, FP and FN.
- `accuracy_score()` calculates overall accuracy.
- `1 - accuracy` calculates error rate.
- `precision_score()` calculates precision.
- `recall_score()` calculates recall.
- `zero_division=0` returns zero instead of a warning if the metric's denominator is zero.

Expected output

Your notebook will display:

```
None
Confusion Matrix:

[[... ...]

 [... ...]]



Accuracy: ...

Error Rate: ...

Precision: ...

Recall: ...
```

The exact values depend on your dataset and model execution.

6. Important viva questions and answers

Question	Answer
What is KNN?	A supervised learning algorithm that classifies data using its nearest neighbors.
What does K represent?	Number of nearest neighbors considered.
What is Outcome?	Target variable representing diabetic or non-diabetic.
What is binary classification?	Classification into two categories.
Why use StandardScaler?	To standardize features before distance calculations.
What is Euclidean distance?	Straight-line distance between two data points.
What is a confusion matrix?	A table comparing actual and predicted classes.
What is accuracy?	Percentage of correct predictions.
What is error rate?	Percentage of incorrect predictions.
What is precision?	Correct positive predictions divided by all predicted positives.
What is recall?	Correct positive predictions divided by all actual positives.

What is a false positive?	Predicting diabetic when the patient is actually non-diabetic.
What is a false negative?	Predicting non-diabetic when the patient actually has diabetes.
What is supervised learning?	Learning from labelled data.
What happens if K is too small?	The model may become sensitive to noise and overfit.
What happens if K is too large?	The model may underfit.

7. Quick revision – ML-3 practical flow

Remember this execution sequence

1. Read dataset – `pd.read_csv()`
2. Separate features and target – `drop()`
3. Split data – `train_test_split()`
4. Normalize – `StandardScaler()`
5. Create KNN model – `KNeighborsClassifier()`
6. Train – `fit()`
7. Predict – `predict()`
8. Evaluate – Confusion Matrix, Accuracy, Error Rate, Precision and Recall

8. Test your understanding

Q1. What does Outcome = 1 represent?

Non-diabetic

Diabetic

Missing value

Q2. Which metric tells us how many actual diabetic patients were correctly identified?

Accuracy

Precision

Recall

Q3. If accuracy is 90%, what is the error rate?

90%

10%

20%

Q4. Why is feature scaling important in KNN?

KNN uses distance calculations

KNN uses decision trees

It removes the target column

Check answers

External viva tip: Remember the five metrics and their differences. In particular, don't confuse precision with recall. Also, this practical is for educational prediction only; it is not a clinical diabetes diagnosis tool.

Prac 4

ML-4: Implement K-Means Clustering on Sales Dataset

Let's understand this practical step by step in simple language, including the Elbow Method, the code, and viva questions.

1. Aim

To implement the K-Means clustering algorithm on the `sales_data_sample.csv` dataset and determine the optimal number of clusters using the Elbow Method.

2. Theory

What is clustering?

Clustering is an unsupervised machine learning technique used to divide data into groups called clusters.

- Data points in the same cluster are similar to each other.
- Data points in different clusters are less similar.
- It does not require predefined output labels.

Example: A company can divide its customers into groups based on their purchasing behaviour:

- Cluster 1: Customers who spend less.
- Cluster 2: Customers who spend moderately.
- Cluster 3: Customers who spend more.

What is K-Means clustering?

K-Means is an unsupervised machine learning algorithm that divides a dataset into K clusters.

Here, K represents the number of clusters we want to create.

Working of K-Means:

1. Select the number of clusters (K).
2. Randomly initialize K centroids (cluster centres).
3. Calculate the distance between each data point and the centroids.

4. Assign each data point to its nearest centroid.
5. Recalculate the centroid of each cluster.
6. Repeat until the clusters stop changing significantly.

K-Means Clustering : What I Learned In Unsupervised Learning | by Aniket | Feb, 2026 | Medium

medium.com

3. What is the Elbow Method?

The Elbow Method is used to find the appropriate number of clusters (K).

We try different values of K, such as 1 to 10, and calculate the Inertia for each value.

What is Inertia?

Inertia measures how close the data points are to their assigned cluster centres.

- Lower inertia means data points are closer to their centroids.
- As the number of clusters increases, inertia generally decreases.
- We select the K value where the decrease in inertia starts becoming slower.

This point looks like an elbow in the graph.

K-means Clustering: Complete Guide with Algorithm, Implementation & Best Practices - Interactive | Michael Brenndoerfer | Michael Brenndoerfer

mbrenndoerfer.com

Remember: The elbow point gives us a reasonable choice for K. It is not always perfectly clear, so we also consider how useful the resulting clusters are.

4. Understanding the dataset

Dataset: `sales_data_sample.csv`

This dataset contains sales transaction information.

Your code selects numerical columns for clustering.

Some important columns:

Column	Meaning
SALES	Total sales amount
QUANTITYORDERED	Quantity ordered
PRICEEACH	Price of each product
ORDERNUMBER	Order identification number
YEAR_ID	Year of the order
MONTH_ID	Month of the order

The code removes identification and date-related columns because they are not useful numerical measurements for this particular clustering approach.

5. Code explanation (line by line)

Step 1: Import libraries

```
None

import pandas as pdimport matplotlib.pyplot as pltfrom
sklearn.preprocessing import StandardScalerfrom sklearn.cluster
import KMeans
```

- `pandas`: Used to read and handle the dataset.
- `matplotlib.pyplot`: Used to draw the elbow graph.
- `StandardScaler`: Used to scale numerical values.
- `KMeans`: Used to perform K-Means clustering.

Step 2: Load the dataset

```
None

df = pd.read_csv(    'sales_data_sample.csv',    encoding='latin1')
```

- `read_csv()` reads the CSV file.
- `encoding='latin1'` helps read special characters present in the dataset.
- `df` stores the dataset as a DataFrame.

Step 3: Select numerical columns

```
None

X = df.select_dtypes(    include='number').drop(    columns=[
'ORDERNUMBER',        'ORDERLINENUMBER',        'QTR_ID',
'MONTH_ID',        'YEAR_ID'    ],    errors='ignore')
```

Let's understand this carefully.

First part:

```
None
df.select_dtypes(include='number')
```

Selects only numerical columns from the dataset.

For example, SALES, PRICEEACH and QUANTITYORDERED.

Second part:

```
None
.drop(columns=[...], errors='ignore')
```

Removes the specified columns.

Why remove them?

- `ORDERNUMBER`: It is an order ID.
- `ORDERLINENUMBER`: It is a line identifier.

- `QTR_ID`: Quarter information.
- `MONTH_ID`: Month information.
- `YEAR_ID`: Year information.

These columns are excluded because their numerical values are identifiers or time codes, not necessarily meaningful measurements of customer similarity.

What does `errors='ignore'` mean?

If any listed column does not exist, Python will ignore it instead of producing an error.

Step 4: Apply StandardScaler

```
None

X = StandardScaler().fit_transform(X)
```

This is a very important step.

Different columns may have different ranges.

For example:

- Quantity: 10 to 100
- Sales: 100 to 10,000
- Price: 20 to 500

If we directly apply K-Means, columns with larger numerical ranges may influence the distance calculation more.

StandardScaler converts values into a common scale.

Formula:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- X = Original value
- μ = Mean
- σ = Standard deviation
- Z = Standardized value

After scaling, each feature approximately has a mean of 0 and standard deviation of 1.

Why is scaling important?

K-Means uses distance calculations. Scaling prevents large-range features from dominating the clustering process.

Step 5: Create an empty list

```
None

inertia = []
```

This list stores the inertia value for every K value.

Step 6: Apply the Elbow Method

```
None

for k in range(1, 11):    model = KMeans(        n_clusters=k,
random_state=42,        n_init=10    )    model.fit(X)
inertia.append(model.inertia_)
```

Let's understand each line.

1. Loop

```
None
for k in range(1, 11):
```

This loop runs 10 times.

The values of K are:

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

2. Create the K-Means model

```
None
model = KMeans(    n_clusters=k,    random_state=42,    n_init=10)
```

- `n_clusters=k`: Number of clusters.
- `random_state=42`: Gives reproducible results.
- `n_init=10`: Runs K-Means 10 times with different initial centroid positions and keeps the best result.

3. Train the model

```
None
model.fit(X)
```

Fits the K-Means model to the dataset and creates clusters.

4. Store inertia

```
None
inertia.append(model.inertia_)
```

Stores the inertia value for the current K in the list.

For example, after the loop, the list contains 10 inertia values.

Step 7: Plot the Elbow Graph

```
None
plt.plot(    range(1, 11),    inertia,
marker='o')plt.xlabel('Number of
clusters')plt.ylabel('Inertia')plt.title('Elbow Method')plt.show()
```

Explanation:

- `plt.plot()`: Draws the graph.
- `range(1,11)`: X-axis represents the number of clusters.
- `inertia`: Y-axis represents inertia values.
- `marker='o'`: Displays circular markers.
- `xlabel()`: Sets the X-axis name.
- `ylabel()`: Sets the Y-axis name.
- `title()`: Gives the graph a title.
- `show()`: Displays the graph.

What do you do after seeing the graph?

Find the elbow point. Your code uses 3 as a simple starting choice, but you should check the actual graph before finalizing K.

Step 8: Create the final K-Means model

```
None
kmeans = KMeans(    n_clusters=3,    random_state=42,    n_init=10)
```

Here, we create the final model with 3 clusters.

```
None
df['Cluster'] = kmeans.fit_predict(X)
```

This is an important line.

- `fit_predict(X)` trains the model and assigns each row to a cluster.
- The assigned cluster number is either 0, 1 or 2.
- `df['Cluster']` creates a new column in the original dataset.

Step 9: Display cluster counts

```
None
print(df['Cluster'].value_counts())
```

Displays the number of records belonging to each cluster.

For example, your output might look like:

```
None
Cluster

0    1200

1     900

2     700
```

Illustrative output only. Your actual cluster counts depend on your dataset and preprocessing.

This means:

- Cluster 0 contains 1200 records.
- Cluster 1 contains 900 records.
- Cluster 2 contains 700 records.

6. Complete code for practical

You can copy this directly into your VS Code Jupyter Notebook.

```
None
import pandas as pdimport matplotlib.pyplot as pltfrom
sklearn.preprocessing import StandardScalerfrom sklearn.cluster
import KMeans# Load datasetdf = pd.read_csv(
'sales_data_sample.csv',    encoding='latin1')# Select numerical
columnsX = df.select_dtypes(include='number').drop(    columns=[
'ORDERNUMBER',        'ORDERLINENUMBER',        'QTR_ID',
'MONTH_ID',        'YEAR_ID'    ],    errors='ignore')# Scale the
dataX = StandardScaler().fit_transform(X)# Elbow Methodinertia =
[]for k in range(1, 11):    model = KMeans(        n_clusters=k,
random_state=42,        n_init=10    )    model.fit(X)
```

7. Important viva questions and answers

Q1. What is clustering?

Clustering is an unsupervised learning technique that groups similar data points together.

Q2. What type of machine learning is K-Means?

Unsupervised machine learning.

Q3. What is K in K-Means?

K represents the number of clusters.

Q4. What is a centroid?

A centroid is the centre point of a cluster, calculated using the mean of the data points belonging to that cluster.

Q5. What is the Elbow Method?

It is a technique used to determine the optimal number of clusters by plotting inertia against different K values.

Q6. What is inertia?

Inertia is the sum of squared distances between data points and their assigned cluster centres.

Q7. Why do we use StandardScaler?

To bring numerical features to a common scale so that large-valued features do not dominate distance calculations.

Q8. What is `fit()`?

It trains the model using the given dataset.

Q9. What is `fit_predict()`?

It trains the model and returns the cluster number assigned to each data point.

Q10. Why do we use `random_state=42`?

To obtain reproducible results.

Q11. What is `n_init=10`?

It runs K-Means with 10 different centroid initializations and selects the best result.

Q12. What are the limitations of K-Means?

- We need to select K.
- It is sensitive to outliers.
- It depends on initial centroid positions.
- It works best for roughly spherical, similarly sized clusters.

Q13. What is the difference between K-Means and hierarchical clustering?

K-Means	**Hierarchical**
Creates K clusters	Creates a hierarchy of clusters

Requires K beforehand	Can explore different cluster counts using a dendrogram
Uses centroids	Uses distances between data points or clusters
Generally faster for large datasets	Can be computationally expensive

8. Quick revision before practical

Remember this sequence:

1. Import libraries
2. Load CSV dataset
3. Select numerical columns
4. Apply StandardScaler
5. Calculate inertia for K = 1 to 10
6. Plot the Elbow Graph
7. Choose K
8. Apply K-Means clustering
9. Display cluster counts

Five keywords to remember:

`Unsupervised Learning` · `Centroid` · `Distance` · `Inertia` · `Elbow Method`

9. Test your understanding

Q1. K-Means is which type of learning?

Supervised

Unsupervised

Reinforcement

Q2. Why do we use StandardScaler?

To remove rows

To scale numerical features

To create labels

Q3. What does inertia measure?

Number of rows

Distance of points from their centroids

Number of columns

Q4. Which method helps select K?

Elbow Method

Decision Tree

Linear Regression

Check answers

One important practical tip: The code clusters individual sales records, not necessarily unique customers. Also, `Cluster 0`, `Cluster 1` and `Cluster 2` are just labels; they do not automatically mean low-, medium- or high-value customers. To give those groups business meaning, examine the average sales and other feature values within each cluster.

Prac 5

ML-5: Implement Gradient Descent Algorithm to Find Local Minima

Let's understand this practical in simple language, step by step, with a dry run so you can explain it confidently during your practical exam.

1. Aim

To implement the Gradient Descent algorithm to find the local minimum of the function:

$$y = (x + 3)^2$$

Starting from $x = 2$.

2. Theory

What is Gradient Descent?

Gradient Descent is an optimization algorithm used to find the minimum value of a function.

In simple words, it tries to find the lowest point of a curve by taking small steps in the direction that decreases the function's value.

For example, imagine a ball rolling down a hill. It keeps moving downward until it reaches the bottom. Gradient Descent works in a similar way.

Medium

medium.com

What is a local minimum?

A local minimum is a point where the function has a lower value than nearby points.

For our function:

$$y = (x + 3)^2$$

The minimum occurs when:

$$x + 3 = 0 \Rightarrow x = -3$$

At $x = -3$, the value of y is 0.

Our algorithm starts at $x = 2$ and gradually moves towards $x = -3$.

3. Important Gradient Descent formula

The main formula is:

$$x_{new} = x_{old} - \alpha f'(x)$$

Where:

- x_{new} = Updated value of x.
- x_{old} = Previous value of x.
- α = Learning rate.
- $f'(x)$ = Derivative or gradient of the function.

For our function:

$$f(x) = (x + 3)^2$$

Its derivative is:

$$f'(x) = 2(x + 3)$$

The derivative tells us the direction and steepness of the curve.

4. Code explanation (line by line)

Step 1: Initialize parameters

```
None

cur_x = 2rate = 0.01precision = 0.000001previous_step_size =
1max_iters = 1000iters = 0
```

Let's understand each variable:

Variable	Meaning
`cur_x = 2`	Starting value of x
`rate = 0.01`	Learning rate
`precision = 0.000001`	Required accuracy / stopping threshold
`previous_step_size = 1`	Initial step size
`max_iters = 1000`	Maximum number of iterations
`iters = 0`	Counts the number of iterations

What is learning rate?

Learning rate decides how big a step the algorithm takes while moving towards the minimum.

- If the learning rate is too high, it may overshoot the minimum.
- If it is too low, the algorithm takes a long time to reach the minimum.

Step 2: Define the derivative

```
None
```

```
df = lambda x: 2 * (x + 3)
```

Here, `lambda` is used to define a small function in one line.

Our original function is:

$$f(x) = (x + 3)^2$$

Its derivative is:

$$f'(x) = 2(x + 3)$$

Therefore, `df` calculates the gradient for any given value of x.

For example, when x = 2:

$$df(2) = 2(2 + 3) = 10$$

So, the gradient is 10.

Step 3: Run the Gradient Descent loop

```
None
```

```
while previous_step_size > precision and iters < max_iters:
```

This loop continues as long as both conditions are true:

1. The previous step size is greater than the required precision.
2. The number of iterations is less than 1000.

When either condition becomes false, the loop stops.

Step 4: Store the previous value

```
None
```

```
prev_x = cur_x
```

Stores the current value of x before updating it.

We need this old value to calculate the next value and the step size.

Step 5: Update x

```
None
```

```
cur_x -= rate * df(prev_x)
```

This is the most important line in the entire program.

It applies the Gradient Descent formula:

$$x_{new} = x_{old} - rate \times f'(x)$$

For the first iteration:

- Previous x = 2
- Learning rate = 0.01
- Gradient = 10

$$x_{new} = 2 - (0.01 \times 10)$$

\[\boxed{x_{\text{new}}=1.9} \]

So, x changes from 2 to 1.9.

Step 6: Calculate step size

```
None
```

```
previous_step_size = abs(prev_x - cur_x)
```

Calculates how much x changed in the current iteration.

For the first iteration:

$$|2 - 1.9| = 0.1$$

`abs()` returns the absolute value, so the step size is always positive.

Step 7: Increase iteration count

```
None

iters += 1
```

Increases the iteration counter by 1.

For example:

- First iteration: 1
- Second iteration: 2
- Third iteration: 3

Step 8: Display the result

```
None

print("Local Minima Occurs at :", cur_x)
```

Displays the final value of x after the loop stops.

5. Dry run with smaller values

Let's manually perform the first few iterations.

Given:

- Starting x = 2
- Learning rate = 0.01
- Gradient = $2(x + 3)$

Iteration	Previous x	Gradient	New x

1	2.0000	10.0000	1.9000
2	1.9000	9.8000	1.8020
3	1.8020	9.6040	1.70596
4	1.70596	9.41192	1.61184

Notice that x keeps decreasing and moves closer to -3.

Gradient descent for neural networks | Lulu's blog

lucidar.me

Important observation: The algorithm does not directly jump from 2 to -3. It takes small steps until it gets sufficiently close to the minimum.

6. Complete Python code

Copy this code into your Jupyter Notebook:

```
None
# Initialize parameterscur_x = 2rate = 0.01precision =
0.000001previous_step_size = 1max_iters = 1000iters = 0# Derivative
of the function (x+3)^2df = lambda x: 2 * (x + 3)# Run Gradient
Descentwhile previous_step_size > precision and iters < max_iters:
prev_x = cur_x    # Update x    cur_x -= rate * df(prev_x)    #
Calculate step size    previous_step_size = abs(prev_x - cur_x)
# Increase iteration count    iters += 1# Display
resultprint("Local Minima Occurs at:", cur_x)print("Number of
iterations:", iters)print("Minimum value of y:", (cur_x + 3)**2)
```

Expected output (approximately):

```
None
Local Minima Occurs at: -2.9999900935

Number of iterations: 650

Minimum value of y: 9.814e-11
```

The exact displayed formatting may differ slightly.

The final x value is approximately -3, and the minimum y value is approximately 0.

7. Viva questions and answers

Q1. What is Gradient Descent?

Gradient Descent is an optimization algorithm used to find the minimum value of a function.

Q2. What is the learning rate?

It controls the size of each step taken towards the minimum.

Q3. What is a gradient?

A gradient is the derivative of a function. It tells us the direction and rate of change of the function.

Q4. Why do we calculate the derivative?

To determine which direction to move to decrease the function's value.

Q5. What is the purpose of `precision`?

It specifies how small the change in x must become before stopping the algorithm.

Q6. Why do we use `abs()`?

To calculate the positive difference between the old and new x values.

Q7. What is the purpose of `max_iters`?

It prevents the loop from running indefinitely.

Q8. What is the minimum of the given function?

For $y = (x + 3)^2$:

- Minimum x = -3
- Minimum y = 0

Q9. What happens if the learning rate is too high?

The algorithm may overshoot the minimum or fail to converge.

Q10. What is the difference between Gradient Descent and K-Means?

Gradient Descent	K-Means
Optimization algorithm	Clustering algorithm
Finds minimum of a function	Groups similar data points
Uses gradients	Uses distances and centroids
Used in model optimization	Used in unsupervised learning

8. Quick revision

Remember these 7 steps:

1. Initialize x, learning rate and precision.
2. Define the derivative.
3. Start the while loop.
4. Store previous x.
5. Update x using the Gradient Descent formula.
6. Calculate step size and increment iterations.
7. Print the minimum.

Five important keywords:

`Optimization` · `Gradient` · `Derivative` · `Learning Rate` · `Local Minimum`

One important concept for your viva: In this example, the function is a simple upward-opening parabola, so its local minimum is also its global minimum. Gradient Descent approaches that minimum from the starting point $x = 2$.